```
4
```

FuelGuard
REST API Reference

Backend: FastAPI · Python 3.11
Database: Cloud Firestore (Firebase)
Storage: Cloudinary
AI: Groq (llama-3.3-70b-versatile)

Base URL

`https://web-production-0cb0e.up.railway.app/api/v1`

Authentication

All protected endpoints require a JWT Bearer token in the `Authorization` header:

```
Authorization: Bearer <access_token>
```

Tokens are obtained via `POST /auth/login` or `POST /auth/signup`. Access tokens expire in **60 minutes**; use `POST /auth/refresh-token` with a refresh token (valid 30 days) to obtain a new one.

GET	Read-only retrieval
POST	Create or trigger action
PUT	Full replacement update
PATCH	Partial update
DELETE	Remove resource
AUTH	JWT token required
ADMIN	Admin/Super-Admin role required
PUBLIC	No authentication required

Contents

1 Authentication `/api/v1/auth`

POST /auth/signup PUBLIC Registers a new user, creates their Firestore profile and mirrors the account into Firebase Auth so email-verification and password-reset emails can be sent. **Request**

Field	Type	Notes
`email`	string	Valid email address
`password`	string	Minimum 8 characters
`full_name`	string	Minimum 2 characters
`phone`	string?	Optional; Pakistani format
`role`	enum	`customer` | `employee` | `admin` | `driver` | `super_admin`

Response

Field	Type	Notes
`access_token`	string	JWT; expires in 60 min
`refresh_token`	string	JWT; expires in 30 days
`token_type`	string	Always `"bearer"`
`uid`	string	Firestore user document ID
`role`	string	Assigned role

POST /auth/login PUBLIC Authenticates a user with email and password. If the bcrypt hash is stale (e.g. after a browser-based Firebase password reset), falls back to Firebase Auth and self-heals the Firestore hash. **Request**

Field	Type	Notes
`email`	string	Registered email
`password`	string	Account password

Response

Field	Type	Notes
`access_token`	string	JWT access token
`refresh_token`	string	JWT refresh token
`token_type`	string	`"bearer"`
`uid`	string	User ID
`role`	string	User role

POST /auth/logout AUTH Stateless logout — the client discards its tokens. No server-side token revocation is performed (add a Firestore denylist for full revocation). **Request** *None (Auth header only)*

Response

Field	Type	Notes
`message`	string	`"Logged out successfully"`

POST /auth/refresh-token PUBLIC Exchanges a valid refresh token for a new access/refresh token pair without requiring the user to log in again. **Request**

Field	Type	Notes
`refresh_token`	string	Valid, non-expired refresh

Response

Field	Type	Notes
`access_token`	string	New JWT access token
`refresh_token`	string	New JWT refresh token
`uid`	string	User ID
`role`	string	User role

POST /auth/forgot-password PUBLIC Triggers a Firebase Auth password-reset email to the given address. Always returns the same message regardless of whether the email is registered, preventing user-enumeration attacks.

Request

Field	Type	Notes
`email`	string	Email to send the reset link to

Response

Field	Type	Notes
`message`	string	Generic confirmation message

POST /auth/reset-password PUBLIC Syncs a Firebase-confirmed password reset back to Firestore by verifying the Firebase ID token and updating the bcrypt hash, then returns fresh JWT tokens so the user is immediately logged in.

Request

Field	Type	Notes
`firebase_id_token`	string	Firebase ID token from `si`
`new_password`	string	New password, minimum

Response

Field	Type	Notes
`message`	string	Confirmation
`access_token`	string	JWT access token
`refresh_token`	string	JWT refresh token
`uid`	string	User ID
`role`	string	User role

POST /auth/chatbot AUTH Sends a user message to the FuelGuard AI support assistant powered by Groq `llama-3.3-70b-versatile`; responds only to FuelGuard-related queries.

Request

Field	Type	Notes
`message`	string	1–1000 characters

Response

Field	Type	Notes
`reply`	string	AI-generated response

2 Users `/api/v1/users`

GET /users/me AUTH Returns the full profile of the currently authenticated user, including verification status and avatar URL. **Request** *None*

Response

Field	Type	Notes
`uid`	string	Unique user ID
`email`	string	Email address
`full_name`	string	Display name
`phone`	string?	Phone number
`role`	enum	User role
`avatar_url`	string?	Cloudinary URL
`is_active`	bool	Account status
`email_verified`	bool	Firebase email verification status
`created_at`	datetime	Registration timestamp

PUT /users/me AUTH Updates the authenticated user's editable profile fields. Only provided fields are updated; omitted fields remain unchanged. **Request**

Field	Type	Notes
`full_name`	string?	New display name
`phone`	string?	New phone number
`avatar_url`	string?	Direct URL (use `POST`

Response *Full `UserProfile` object (same as GET /users/me)*

POST /users/me/avatar AUTH Uploads a profile picture as a multipart file, stores it on Cloudinary under `fuelguard/avatars/{uid}/` with a 512×512 face-crop transform, and saves the returned URL to Firestore. **Request**

Field	Type	Notes
`file`	image (multipart)	JPEG/PNG profile picture

Response *Full `UserProfile` object with updated `avatar_url`*

PUT /users/me/password AUTH Changes the authenticated user's password after verifying the current one against the Firestore bcrypt hash. **Request**

Field	Type	Notes
`current_password`	string	Existing password
`new_password`	string	New password, minim

Response

Field	Type	Notes
`message`	string	`"Password changed successfully"`

GET /users ADMIN Lists all registered users with optional filters. Results are paginated using limit/offset and password hashes are stripped from every response. **Request** *Query params:* `role`, `is_active`, `limit` (max 200), `offset`

Response

Field	Type	Notes
`total`	int	Total matching users
`items`	array	Array of user profile objects

GET /users/{user_id} ADMIN Returns the full profile of any user by their Firestore document ID; password hash is stripped from the response. **Request** *Path param:* `user_id`

Response *User profile object*

PUT /users/{user_id}/role ADMIN Reassigns a user to a different role. Use with caution — role changes take effect immediately on the next token refresh.

Request

Field	Type	Notes
`role`	enum	`customer` | `employee`

Response

Field	Type	Notes
`message`	string	Confirmation
`role`	string	Newly assigned role

PATCH /users/{user_id}/status ADMIN Activates or deactivates a user account. Deactivated users receive a 403 on login until reactivated.

Request

Field	Type	Notes
`is_active`	bool	`true` to activate, `false` to deact

Response

Field	Type	Notes
`message`	string	Confirmation
`is_active`	bool	New status

DELETE /users/{user_id} ADMIN Permanently deletes a user from Firestore. Admins cannot delete their own account; 204 is returned on success with no response body.

Request *Path param:* `user_id`

Response *204 No Content*

3 Sessions `/api/v1/sessions`

POST /sessions/generate-qr ADMIN Generates a session QR code for a specific nozzle. The QR encodes a signed session ID and has a configurable expiry (default 5 minutes). **Request**

Field	Type	Notes
`nozzle_id`	string	Target nozzle ID
`expires_in_seconds`	int	Default 300

Response

Field	Type	Notes
`session_id`	string	New session ID
`qr_data`	string	Raw data encoded in the QR
`qr_image_base64`	string	Base64 PNG of the QR image
`expires_at`	datetime	Expiry timestamp

POST /sessions/scan AUTH Processes a scanned QR code and activates the corresponding session, linking it to the authenticated user or a provided user ID. **Request**

Field	Type	Notes
`qr_data`	string	Raw QR payload
`user_id`	string?	Optional override

Response

Field	Type	Notes
`session_id`	string	Activated session ID
`status`	string	`"active"`
`nozzle_id`	string	Associated nozzle

POST /sessions/start AUTH Device-initiated session start — called by the mobile app after connecting to the nozzle's WiFi hotspot (SSID: FuelMonitor) via QR nozzle ID. **Request**

Field	Type	Notes
`nozzle_id`	string	Nozzle to start the session on

Response

Field	Type	Notes
`session_id`	string	New session ID
`nozzle_id`	string	Confirmed nozzle ID
`status`	string	`"active"`

GET /sessions/active ADMIN Returns all sessions currently in `active` status. Used by the admin dashboard to monitor live fueling activity across all nozzles. **Request** *None*

Response *Array of session objects*

GET /sessions/concurrent ADMIN Checks how many concurrent sessions are running on a specific nozzle. Used for fraud detection and capacity management. **Request** *Query param:* `nozzle_id`

Response

Field	Type	Notes
`nozzle_id`	string	Queried nozzle
`concurrent_count`	int	Number of active sessions
`sessions`	array	Session IDs

GET /sessions/{session_id} AUTH Returns full details of a session including status, timestamps, total litres dispensed, total amount, and linked transaction ID. **Request** *Path param:* `session_id`

Response

Field	Type	Notes
`id`	string	Session ID
`nozzle_id`	string	Nozzle ID
`user_id`	string	Linked user
`status`	enum	`active | completed | failed | pending`
`started_at`	datetime	Start timestamp
`ended_at`	datetime?	End timestamp
`total_litres`	float	Litres dispensed
`total_amount`	float	Amount in PKR
`transaction_id`	string?	Created transaction ID

POST /sessions/{session_id}/close AUTH Manually closes an active or pending session. Accepts an optional reason string; defaults to `"manual"`. **Request**

Field	Type	Notes
`reason`	string?	Closure reason, default `"`

Response

Field	Type	Notes
`message`	string	Confirmation

POST /sessions/timeout-check PUBLIC Background job endpoint that auto-closes sessions that have exceeded their expiry time without activity. Called by a scheduler. **Request** *None*

Response

Field	Type	Notes
`timed_out_sessions_closed`	int	Count of sessions closed

4 Transactions `/api/v1/transactions`

POST /transactions AUTH Records a completed fuel dispensing transaction with litres, price, and payment method. Fraud detection runs asynchronously on the created record. **Request**

Field	Type	Notes
`session_id`	string	Linked session
`nozzle_id`	string	Nozzle used
`user_id`	string	Customer
`fuel_type`	enum	`petrol` | `diesel` | `cng` | `lpg`
`litres_dispensed`	float	Must be >0
`price_per_litre`	float	PKR per litre
`payment_method`	enum	`cash` | `card` | `wallet` | `qr_pay`
`vehicle_id`	string?	Optional fleet vehicle
`employee_id`	string?	Optional serving employee

Response

Field	Type	Notes
`id`	string	Transaction ID
`total_amount`	float	`litres` × `price` in PKR
`status`	enum	`completed` | `pending` | `failed`
`is_flagged`	bool	Fraud flag status
`created_at`	datetime	Timestamp

GET /transactions/my AUTH Returns the authenticated user's own transaction history, paginated. Includes completed, pending and failed records. **Request** *Query params:* `limit` (max 100, default 20), `offset`

Response

Field	Type	Notes
`total`	int	Total transactions for this user
`items`	array	Paginated transaction objects

GET /transactions/{transaction_id} AUTH Returns the full detail of a single transaction including receipt URL, evidence URL, and fraud flag status. **Request** *Path param:* `transaction_id`

Response *Full transaction object*

GET /transactions/{transaction_id}/receipt AUTH Generates and streams a PDF receipt for the transaction. Receipt includes transaction details, station info, and a unique reference number. **Request** *Path param:* `transaction_id`

Response *PDF file stream (`application/pdf`)*

GET /transactions ADMIN Lists all transactions with optional filters. Results include fraud flags and are paginated. Password fields are never returned. **Request** *Query params:* `nozzle_id`, `user_id`, `station_id`, `fuel_type`, `limit`, `offset`

Response

Field	Type	Notes
`total`	int	Total matching records
`items`	array	Transaction objects

POST /transactions/{transaction_id}/price-update ADMIN Adjusts the price-per-litre of an existing transaction and recalculates the total amount. The reason for the update is stored for audit.

Request

Field	Type	Notes
`price_per_litre`	float	New corrected price
`reason`	string?	Audit note

Response

Field	Type	Notes
`message`	string	Confirmation
`new_total_amount`	float	Recalculated total in PKR

5 Stations `/api/v1/stations`

GET /stations/nearby AUTH Returns fuel stations within a given radius of the provided coordinates, sorted by distance. Optionally filtered by fuel type. **Request** *Query params:* `latitude`, `longitude`, `radius_km` (default 10), `fuel_type?`

Response *Array of station objects with* ***`distance_km`*** *field added*

GET /stations/route AUTH Finds stations along a driving route by calculating a midpoint buffer between origin and destination coordinates. **Request** *Query params:* `origin_lat`, `origin_lng`, `dest_lat`, `dest_lng`, `fuel_type?`

Response *Array of stations with* ***`distance_from_route_km`***

GET /stations/me/favorites AUTH Returns the authenticated user's list of favourite stations with current prices and operating hours. **Request** *None*

Response *Array of station objects*

POST /stations/me/favorites/{station_id} AUTH Adds a station to the current user's favourites list in Firestore. Duplicate additions are silently ignored. **Request** *Path param:* `station_id`

Response

Field	Type	Notes
`message`	string	Confirmation

DELETE /stations/me/favorites/{station_id} AUTH Removes a station from the user's favourites list. Returns 204 with no body on success. **Request** *Path param:* `station_id`

Response *204 No Content*

POST /stations ADMIN Creates a new fuel station record with location, contact info, and available fuel types. Station is active by default. **Request**

Field	Type	Notes
`name`	string	Station display nam
`address`	string	Full street address
`city`	string	City
`latitude`	float	GPS latitude
`longitude`	float	GPS longitude
`fuel_types_available`	array	e.g. `["petrol","di`
`contact_phone`	string?	Optional
`manager_uid`	string?	Optional manager
`operating_hours`	string?	e.g. `"24/7"`

Response *Full station object with generated* ***`id`***

GET /stations/{station_id} AUTH Returns a single station's full profile including current fuel prices fetched live from the prices collection. **Request** *Path param:* `station_id`

Response *Station object with* ***`current_prices`*** *array*

PUT /stations/{station_id} ADMIN Updates any editable field of an existing station. Partial updates are supported — only provided fields are overwritten. **Request** *Any subset of station creation fields*

Response *Updated station object*

DELETE /stations/{station_id} ADMIN Permanently removes a station from Firestore. Associ-

ated nozzles should be reassigned or deleted beforehand. Returns 204 on success. **Request**

Path param: `station_id`

Response *204 No Content*

6 Prices `/api/v1/prices`

GET /prices/compare AUTH Compares the current price of a given fuel type across all stations, optionally filtered to those within a radius of the user's location. **Request** *Query params:* `fuel_type` (required), `latitude?`, `longitude?`, `radius_km` (default 20)

Response

Field	Type	Notes
`station_id`	string	Station ID
`station_name`	string	Station name
`price_per_litre`	float	Current price in PKR
`distance_km`	float?	Distance if coords provided

GET /prices/cheapest AUTH Finds the single cheapest station for a fuel type within the specified radius. Returns full station info and distance for navigation. **Request** *Query params:* `fuel_type`, `latitude`, `longitude`, `radius_km` (default 20)

Response *Single station object with* ***`price_per_litre`*** *and* ***`distance_km`***

GET /prices/alerts AUTH Lists all price-drop alerts the authenticated user has set up across any station and fuel type combination. **Request** *None*

Response *Array of alert objects*

POST /prices/alerts AUTH Creates a price alert that notifies the user when the price at a given station drops below the specified target. **Request**

Field	Type	Notes
`station_id`	string	Target station
`fuel_type`	enum	Fuel type to watch
`target_price`	float	Alert threshold in PKR

Response

Field	Type	Notes
`id`	string	Alert ID
`target_price`	float	Threshold
`is_active`	bool	Active status
`created_at`	datetime	Creation timestamp

DELETE /prices/alerts/{alert_id} AUTH Permanently removes a price alert. Returns 204 with no body. **Request** *Path param:* `alert_id`

Response *204 No Content*

POST /prices/{station_id}/update ADMIN Updates the current fuel price at a station. The previous price is archived in the price history collection for trend analysis. **Request** *Path param:* `station_id` · *Query params:* `fuel_type`, `price_per_litre`

Response

Field	Type	Notes
`message`	string	Confirmation
`fuel_type`	string	Updated fuel type
`price_per_litre`	float	New price in PKR

GET /prices/{station_id}/history AUTH Returns the historical price records for a station, optionally filtered by fuel type. Useful for trend charts. **Request** *Path param:* `station_id`

· *Query param:* `fuel_type?`

Response *Array of price history objects ordered by date*

7 Nozzles `/api/v1/nozzles`

GET /nozzles AUTH Lists all registered nozzles with optional filters for station, operational status, and fuel type. Returns live readings alongside static config. **Request** *Query params:* `station_id?`, `status?`, `fuel_type?`

Response *Array of NozzleResponse objects*

POST /nozzles ADMIN Registers a new IoT nozzle device with its BLE identifiers, hardware serial, and flow meter configuration. Status defaults to `idle`. **Request**

Field	Type	Notes
`name`	string	Nozzle displa
`station_id`	string	Parent static
`fuel_type`	enum	`petrol | di`
`ble_uuid`	string	Bluetooth LI
`ble_device_name`	string	BLE advertis
`hardware_serial`	string	Physical seri
`flow_meter_model`	string?	Optional mo
`max_flow_rate`	float	L/min, defau

Response *Full NozzleResponse object with generated* ***id***

GET /nozzles/{nozzle_id} AUTH Returns the full profile and live reading of a nozzle, including tamper detection status, current flow rate, and last-seen timestamp. **Request** *Path param:* `nozzle_id`

Response

Field	Type	Notes
`id`	string	Nozzle ID
`status`	enum	`idle | active | error | maintenance`
`tamper_detected`	bool	Hardware tamper flag
`current_flow_rate`	float	Current L/min
`total_litres_today`	float	Today's dispensed total
`last_seen`	datetime	Last telemetry timestamp

PUT /nozzles/{nozzle_id} ADMIN Updates editable fields of a nozzle such as name, fuel type, flow meter model, and max flow rate. Partial updates supported. **Request** *Any subset of creation fields except* ***station_id*** *and* ***ble_uuid***

Response *Updated NozzleResponse object*

DELETE /nozzles/{nozzle_id} ADMIN Permanently deletes a nozzle record from Firestore. Ensure all active sessions on this nozzle are closed before deletion. **Request** *Path param:* `nozzle_id`

Response *204 No Content*

POST /nozzles/{nozzle_id}/readings PUBLIC Accepts real-time flow meter readings pushed by the ESP32 device over WiFi. Updates the nozzle's live telemetry in Firestore. **Request**

Field	Type	Notes
`flow_rate`	float	Current flow in L/min
`total_dispensed`	float	Cumulative litres this session

Response

Field	Type	Notes
`status`	string	`"ok"`

POST /nozzles/{nozzle_id}/tamper-alert PUBLIC Reports a hardware tamper event from the IoT device. Creates a tamper alert record and sets the nozzle's tamper flag to `true`.

Request

Field	Type	Notes
`alert_type`	string	Type of tamper event
`description`	string?	Optional details
`timestamp`	datetime?	Event time from device

Response

Field	Type	Notes
`alert_id`	string	Created alert ID
`message`	string	Confirmation

PATCH /nozzles/alerts/{alert_id}/resolve ADMIN Marks a tamper alert as resolved and stores the resolution note for the audit trail. Resets the nozzle's tamper flag.

Request

Field	Type
`resolution_note`	string

Response

Field	Type	Notes
`message`	string	Confirmation

8 Evidence `/api/v1/evidence`

POST /evidence AUTH Uploads a photo captured at the pump to Cloudinary and stores the URL in the evidence collection. A thumbnail URL is generated automatically via Cloudinary transformations. Evidence is auto-deleted after the retention period (90 days). **Request** *Query params:* `transaction_id`, `nozzle_id`, `session_id?`, `capture_trigger` (default `"auto"`) · Body: `file` (multipart image)

Response

Field	Type	Notes
`id`	string	Evidence ID
`image_url`	string	Full-size Cloudinary URL
`thumbnail_url`	string	320×320 thumbnail URL
`watermark_text`	string	Embedded transaction reference
`delete_at`	datetime	Auto-delete timestamp

GET /evidence/{transaction_id} AUTH Returns all evidence records linked to a transaction. A single transaction may have multiple photos from different capture triggers. **Request** *Path param:* `transaction_id`

Response *Array of evidence objects*

GET /evidence/{evidence_id}/image AUTH Returns both the full-size and thumbnail Cloudinary URLs for a specific evidence record without returning the full metadata. **Request** *Path param:* `evidence_id`

Response

Field	Type	Notes
`image_url`	string	Full-size image URL
`thumbnail_url`	string?	Thumbnail URL

DELETE /evidence/{evidence_id} ADMIN Deletes the evidence from both Cloudinary and Firestore. Used for manual removal before the automatic retention period expires. **Request** *Path param:* `evidence_id`

Response *204 No Content*

POST /evidence/auto-delete PUBLIC Background job that enforces the retention policy by deleting all evidence records whose `delete_at` timestamp has passed from both Cloudinary and Firestore. **Request** *None*

Response

Field	Type	Notes
`deleted_count`	int	Number of records deleted

GET /evidence ADMIN Lists all evidence with optional filters by nozzle or capture trigger. Results are paginated and ordered by creation date descending. **Request** *Query params:* `nozzle_id?`, `capture_trigger?`, `limit` (max 200), `offset`

Response

Field	Type	Notes
`total`	int	Total matching records
`items`	array	Evidence objects

9 Fraud Detection `/api/v1/fraud`

GET /fraud/alerts ADMIN Lists fraud alerts with filters for type, status, and severity. All alerts are paginated and include the associated transaction and nozzle IDs. **Request** *Query params:* `alert_type?`, `status?`, `severity?`, `limit`, `offset`

Response

Field	Type	Notes
`total`	int	Total matching alerts
`items`	array	Fraud alert objects

POST /fraud/alerts/{alert_id}/resolve ADMIN Resolves an open fraud alert by recording the resolution status and an explanatory note. Resolved alerts are retained for audit history. **Request**

Field	Type	Notes
`status`	enum	`resolved | dismissed | escalated`
`resolution_note`	string	Explanation of resolution

Response

Field	Type	Notes
`message`	string	Confirmation

POST /fraud/analyze ADMIN Runs the fraud analysis pipeline on a specific transaction and generates any alerts triggered by flow discrepancies or anomalous patterns. **Request** *Query param:* `transaction_id`

Response

Field	Type	Notes
`transaction_id`	string	Analyzed transaction
`alerts_generated`	int	Number of new alerts
`alerts`	array	Generated alert objects

GET /fraud/stats ADMIN Returns aggregated fraud statistics including open alert count, alerts resolved this week, and breakdowns by type and severity. **Request** *None*

Response

Field	Type	Notes
`total_alerts`	int	All-time alert count
`open_alerts`	int	Currently open
`resolved_this_week`	int	Resolved in the last 7 days
`by_type`	dict	Alert counts by type
`by_severity`	dict	Alert counts by severity

POST /fraud/blacklist ADMIN Adds a user, vehicle, or device entity to the fraud blacklist with a reason. Blacklisted entities are flagged on future transactions automatically. **Request**

Field	Type	Notes
`entity_type`	enum	`user | vehicle | device`
`entity_id`	string	ID of the entity to blacklist
`reason`	string	Reason for blacklisting
`flagged_by`	string?	Optional admin user ID

Response *Blacklist entry object with generated* `id`

POST /fraud/flag AUTH Manually flags a transaction as suspicious with a custom reason and severity. Creates a fraud alert that appears in the admin dashboard.

Request

Field	Type	Notes
`transaction_id`	string	Transaction to flag
`reason`	string	Description of suspicion
`severity`	enum	`low` | `medium` | `high` | `critical`, default `medium`

Response

Field	Type	Notes
`alert_id`	string	Created alert ID
`message`	string	Confirmation

10 Fleet Management `/api/v1/fleet`

GET /fleet/vehicles AUTH Lists vehicles owned by the current user. Admins see all vehicles. Includes total fuel consumed and total expense aggregates. **Request** *None*

Response *Array of vehicle objects*

POST /fleet/vehicles AUTH Registers a new fleet vehicle with its registration details, fuel type, and tank capacity. Optionally assigns an owner user ID. **Request**

Field	Type	Notes
`registration_number`	string	Unique
`make`	string	Manufac
`model`	string	Model n
`year`	int	Model y
`fuel_type`	enum	Fuel typ
`tank_capacity`	float	Tank siz
`owner_uid`	string?	Optiona

Response *Full vehicle object with generated* `id`

GET /fleet/vehicles/{vehicle_id}/consumption AUTH Returns aggregated fuel consumption data for a vehicle including a monthly breakdown of litres dispensed and cost in PKR. **Request** *Path param:* `vehicle_id`

Response

Field	Type	Notes
`total_fuel_consumed`	float	All-time litres
`monthly_breakdown`	array	Monthly litres and cost

POST /fleet/expenses AUTH Logs a fleet expense (fuel, maintenance, etc.) against a vehicle. Fuel expenses automatically update the vehicle's `total_fuel_consumed` counter. **Request**

Field	Type	Notes
`vehicle_id`	string	Target vehicle
`category`	enum	`fuel` | `maintenance` | `insurance` | `other`
`amount`	float	Cost in PKR
`litres`	float?	If fuel category
`station_id`	string?	Refuelling station
`description`	string?	Notes
`expense_date`	date?	Defaults to today

Response *Full expense object with generated* `id`

GET /fleet/budget AUTH Returns the monthly budget for the current user (optionally per vehicle), including spent amount and remaining balance. **Request** *Query params:* `vehicle_id?`, `month?` (1–12), `year?`

Response

Field	Type	Notes
`budget_amount`	float	Set budget in PKR
`spent_amount`	float	Total spent this period
`remaining`	float	Remaining budget

PUT /fleet/budget AUTH Sets or updates the monthly fuel budget for a user or specific vehicle.

Overwrites any existing budget for the same period. **Request**

Field	Type	Notes
`month`	int	1–12
`year`	int	e.g. 2025
`amount`	float	Budget in PKR
`vehicle_id`	string?	Optional; applies to sp

Response

Field	Type	Notes
`message`	string	Confirmation
`budget_id`	string	Budget document ID

PUT /fleet/vehicles/{vehicle_id}/driver AUTH Assigns a driver to a vehicle by linking the driver's UID. A vehicle can have only one assigned driver at a time. **Request**

Field	Type	Notes
`driver_uid`	string	Driver u

Response

Field	Type	Notes
`message`	string	Confirmation

11 IoT Devices `/api/v1/iot`

POST /iot/devices/register ADMIN Registers a new IoT device (ESP32, flow meter, camera, or gateway) and links it to a nozzle and station. Sets initial firmware version and status. **Request**

Field	Type	Notes
`device_id`	string	Unique hardware device ID
`device_type`	enum	`esp32 | flow_meter | camera | gateway`
`firmware_version`	string	Current firmware version
`nozzle_id`	string?	Linked nozzle
`station_id`	string?	Parent station
`ip_address`	string?	Local IP

Response *Full IoT device object with generated* `id`

POST /iot/devices/{device_id}/ping PUBLIC Device heartbeat endpoint called by the ESP32 on a regular interval to report it is alive. Updates `last_ping` and device status in Firestore. **Request**

Field	Type	Notes
`firmware_version`	string?	Optional version report
`ip_address`	string?	Current IP

Response

Field	Type	Notes
`status`	string	`"ok"`
`server_time`	datetime	Server UTC time for clock sync

POST /iot/devices/{device_id}/telemetry PUBLIC Accepts sensor telemetry packets from the device including flow rate, tamper detection, temperature, and signal strength for logging. **Request**

Field	Type	Notes
`nozzle_id`	string?	Reported nozzle
`flow_rate`	float?	L/min
`total_dispensed`	float?	Session cumulative litres
`tamper_detected`	bool?	Hardware tamper flag
`temperature`	float?	Sensor temperature °C
`signal_strength`	int?	WiFi RSSI dBm
`timestamp`	datetime?	Device-side timestamp

Response

Field	Type	Notes
`status`	string	`"ok"`

POST /iot/firmware/upload ADMIN Uploads a firmware binary file to Cloudinary (`resource_type: raw`) and creates a firmware release record. Devices pull this URL during OTA updates. **Request** *Query params:* `version`, `release_notes?`, `is_mandatory` (default false) · Body: `file` (binary .bin)

Response

Field	Type	Notes
`id`	string	Firmware record ID
`version`	string	Version string
`download_url`	string	Cloudinary URL for the binary
`is_mandatory`	bool	Whether devices must update

POST /iot/devices/{device_id}/firmware/update ADMIN Queues a firmware update command for a specific device. The device picks up the command on its next ping and initiates the OTA download.

Request

Field	Type	Notes
`firmware_version`	string	Target version to install
`force`	bool	Override if already on this version

Response

Field	Type	Notes
`command_id`	string	Queued command ID
`message`	string	Confirmation

PUT /iot/devices/{device_id}/calibrate ADMIN Queues a flow meter calibration command with a new calibration factor and offset. Applied by the device on the next maintenance window.

Request

Field	Type	Notes
`calibration_factor`	float	Multiplier for raw readings
`offset`	float	Additive offset, default 0

Response

Field	Type	Notes
`command_id`	string	Queued command ID
`message`	string	Confirmation

12 Admin Dashboard `/api/v1/admin`

GET /admin/overview ADMIN Returns a real-time dashboard snapshot with active session count, today's revenue and transaction count, and open fraud alerts. **Request** *None*

Response

Field	Type	Notes
`active_sessions`	int	Currently running sessions
`revenue_today_pkr`	float	Today's total revenue
`transactions_today`	int	Today's transaction count
`open_fraud_alerts`	int	Unresolved alerts
`avg_litres_per_transaction`	float	Average litres
`generated_at`	datetime	Snapshot timestamp

GET /admin/nozzles/live ADMIN Returns live operational status of all nozzles, including current flow rate, tamper flags, and last-seen timestamps for real-time monitoring. **Request** *None*

Response *Array of nozzle objects with live readings*

POST /admin/employees ADMIN Creates a new employee account with the `employee` role. The account is immediately active and can log in with the provided credentials. **Request**

Field	Type	Notes
`email`	string	Employee email
`password`	string	Minimum 8 characters
`full_name`	string	Minimum 2 characters
`phone`	string?	Optional

Response *UserProfile object for the new employee*

GET /admin/analytics ADMIN Returns aggregated business analytics for the current period including revenue by fuel type and transaction breakdown by payment method. **Request** *None*

Response

Field	Type	Notes
`total_transactions`	int	Period total
`total_revenue_pkr`	float	PKR revenue
`total_litres_dispensed`	float	Litres dispensed
`revenue_by_fuel_type`	dict	Breakdown per fuel type
`transactions_by_payment`	dict	Breakdown per payment method
`open_fraud_alerts`	int	Current open alerts

POST /admin/complaints ADMIN Logs a customer complaint against a user ID with subject and description. Status defaults to `open` and appears in the complaints queue. **Request**

Query params: `user_id`, `subject`, `description`

Response

Field	Type	Notes
`id`	string	Complaint ID
`status`	string	`"open"`
`created_at`	datetime	Timestamp

PATCH /admin/complaints/{complaint_id} ADMIN Updates a complaint's status and optionally adds a resolution note. Status transitions: `open` → `in_progress` → `resolved | closed`.

Request *Query params:* `status`, `resolution_note?`

Response

Field	Type	Notes
`message`	string	Confirmation

13 Reports `/api/v1/reports`

GET /reports/transactions ADMIN Generates a transaction report for the selected period. Supports daily, weekly, and monthly aggregation with optional station and fuel type filters. **Request** *Query params:* `period` (`daily|weekly|monthly`), `station_id?`, `fuel_type?`

Response *Report object with transaction list and aggregates*

GET /reports/sales ADMIN Returns a sales summary report grouped by the specified period. Includes total revenue, litres, and transaction count per period segment. **Request** *Query params:* `period`, `station_id?`

Response *Sales report with period breakdowns*

GET /reports/fraud ADMIN Produces a fraud report for the period showing alert counts by type and severity, useful for compliance and security reviews. **Request** *Query param:* `period` (default `monthly`)

Response

Field	Type	Notes
`period`	string	Report period
`total`	int	Total alerts
`by_type`	dict	Alert counts per type
`items`	array	Individual alert records

GET /reports/employees ADMIN Reports employee performance for the period, showing each employee's transaction count, total revenue handled, and litres dispensed. **Request** *Query param:* `period` (default `monthly`)

Response

Field	Type	Notes
`period`	string	Report period
`employees`	dict	Keyed by employee ID: `transaction_count`, `total_revenue`, `total_litres`

GET /reports/nozzles ADMIN Returns nozzle performance metrics for the period. Optionally filtered by station. Useful for identifying underperforming or overloaded nozzles. **Request** *Query params:* `period`, `station_id?`

Response

Field	Type	Notes
`period`	string	Report period
`nozzles`	dict	Keyed by nozzle ID: `transaction_count`, `total_revenue`, `total_litres`

GET /reports/comparative ADMIN Compares a metric (revenue, litres, or transactions) between two periods and returns the percentage change. **Request** *Query params:* `metric` (`revenue|litres|transactions`), `current_period`, `previous_period`

Response

Field	Type	Notes
`metric`	string	Compared metric
`current`	object	`total` and `count` for current period
`previous`	object	`total` and `count` for previous period
`change_percent`	float	Percentage change

POST /reports/export ADMIN Exports a report as a downloadable file. Supports PDF (ReportLab), Excel (openpyxl), and CSV formats streamed directly in the response. **Request** *Query params:* `report_type` (`transactions|sales|fraud|employees`), `format` (`pdf|excel|csv`), `period`

Response *File download (`application/pdf`, `.xlsx`, or `.csv`)*

GET /reports/charts/transactions ADMIN Returns daily transaction chart data (count, revenue, litres per day) for the requested period, ready to be consumed by a charting library. **Request** *Query param:* `period` (default `weekly`)

Response

Field	Type	Notes
`period`	string	Chart period
`data`	dict	Keyed by `YYYY-MM-DD`: `count`, `revenue`, `litres`

GET /reports/charts/revenue ADMIN Returns daily revenue figures for the chart period. Keyed by date string for direct mapping to time-series chart axes. **Request** *Query param:* `period`

Response

Field	Type	Notes
`revenue_by_day`	dict	`YYYY-MM-DD` → PKR float

GET /reports/charts/fraud ADMIN Returns fraud alert distribution data by type and severity for the period, suitable for pie or bar chart visualisation on the admin dashboard. **Request** *Query param:* `period` (default `monthly`)

Response

Field	Type	Notes
`by_type`	dict	Alert count per type
`by_severity`	dict	Alert count per severity

Appendix — HTTP Status Codes

Code	Meaning
`200 OK`	Request succeeded; response body contains result
`201 Created`	Resource created; response body contains the new object
`204 No Content`	Success with no response body (DELETE operations)
`400 Bad Request`	Invalid input, missing required field, or validation error
`401 Unauthorized`	Missing or invalid JWT token
`403 Forbidden`	Valid token but insufficient role permissions
`404 Not Found`	Requested resource does not exist
`422 Unprocessable`	Request body schema validation failed (Pydantic)
`429 Too Many Requests`	Rate limit exceeded (AI endpoints)
`500 Internal Error`	Unexpected server-side failure
`502 Bad Gateway`	Upstream service failure (Cloudinary, Firebase, Groq)
`503 Unavailable`	Required third-party credentials not configured

Appendix — Base Firestore Collections

Collection	Purpose
`users`	User profiles, roles, hashed passwords, avatar URLs
`nozzles`	IoT nozzle configs and live telemetry
`sessions`	Fueling session lifecycle records
`transactions`	Completed dispense records with fraud flags
`evidence`	Photo metadata and Cloudinary URLs
`fraud_alerts`	Detected and manually flagged anomalies
`stations`	Fuel station master data
`prices`	Current fuel prices per station
`price_history`	Historical price records for trend analysis
`fleet_vehicles`	Registered fleet vehicles
`fleet_expenses`	Fuel and maintenance expense logs
`fleet_drivers`	Driver profiles and vehicle assignments
`iot_devices`	Hardware device registry
`iot_firmware`	Firmware release records
`blacklist`	Blacklisted users, vehicles, and devices
`complaints`	Customer complaint records
`notifications`	User notification queue
`price_alerts`	User-set price-drop alerts
`budgets`	Monthly fleet budget records
`tamper_alerts`	Hardware tamper event records
`device_commands`	Queued OTA and calibration commands
`telemetry_logs`	Raw IoT sensor telemetry archive